NeuroCamo

100 Expected Viva Questions

With model answers - every figure measured, none estimated

Format	100 questions with model answers, grouped by topic
Sections	Basics / SNN theory / architecture / data / training / results / comparison / dashboard / code / hard questions
Use	The hard-questions section is where marks are won or lost
Rule	Never invent a number. Every figure here is measured.

A. Project basics (Q1-Q12)

Q1. What is your project in one sentence?

A camouflaged-object detector built on a spiking neural network: it takes an image, converts it into spike trains, and outputs a bounding box plus a confidence that a concealed target is present, using only 0.33 M parameters.

Q2. What is camouflaged object detection, and why is it harder than normal detection?

The target's colour, texture and outline are deliberately matched to the background. Ordinary detectors rely on boundary contrast; camouflage destroys exactly that cue, so the model must reason about subtle texture discontinuity instead of edges.

Q3. Why did you choose this problem?

It has real applications - military reconnaissance, wildlife survey, search and rescue - and it is a case where the standard detector assumption breaks, which makes it a genuine research question rather than a re-implementation.

Q4. What are the practical applications?

Perimeter surveillance, drone reconnaissance, camouflaged-animal census, defect detection on textured surfaces, and polyp or lesion detection in medical imaging, which is structurally the same low-contrast problem.

Q5. Why a spiking neural network and not a normal CNN?

Energy. Spiking neurons only compute when they receive a spike, so on event-driven hardware the cost scales with activity rather than with layer size. Our target platform is a battery-powered edge device, which makes that trade-off the point of the project.

Q6. What is your final accuracy?

38.6% mean IoU and 33.7% IoU@50 on the held-out test split, against a mean-box baseline of 37.0%, with a confidence separation of +0.288, after four epochs.

Q7. How many parameters does your model have?

329,096 in total. The spiking backbone alone is 93 K; the rest is the attention bridge and the dense head.

Q8. What hardware did you train on?

A single NVIDIA RTX 3050 Ti Laptop GPU with 4 GB of memory, PyTorch 2.5.1 with CUDA 12.1, and snnTorch for the spiking layers.

Q9. How long does inference take?

About 105 ms per frame end to end, including Poisson encoding and all 16 timesteps.

Q10. Is that fast enough for real time?

About 9-10 frames per second, so it is usable for surveillance but not for high-speed tracking. Latency is dominated by running 16 sequential timesteps, which is the price of the spiking formulation on a GPU.

Q11. What is genuinely novel here?

Applying an attention-based spiking pipeline to camouflaged detection with mask-density supervision on the heatmap, and demonstrating measurably that supervision density - not architecture - was the limiting factor at this parameter scale.

Q12. Who did what on the team?

Answer this honestly and specifically for your own team - dataset preparation, backbone and bridge implementation, training and evaluation, and the web dashboard were the four workstreams.

B. Spiking neural network theory (Q13-Q30)

Q13. What is a spiking neuron?

A unit with internal state. It integrates incoming current into a membrane potential, and when that potential crosses a threshold it emits a single binary spike and resets. Information is carried by when and how often it fires, not by a real-valued activation.

Q14. Write the LIF equation you used.

V(t+1) = beta * V(t) + W . x(t) - V_th * S(t), where S(t) = 1 if V(t) >= V_th else 0. The first term is leak, the second is input current, the third is reset-by-subtraction.

Q15. What is beta and what value did you use?

The membrane decay factor - how much potential survives to the next timestep. We initialise it at 0.9 and make it learnable, so each layer settles on its own time constant.

Q16. What happens if beta is 0, or 1?

At 0 there is no memory at all and the network becomes a stateless per-timestep classifier. At 1 nothing ever leaks, so potential accumulates forever and every neuron eventually fires regardless of input.

Q17. What is the threshold, and did you learn it?

V_th is the firing threshold, kept at the snnTorch default of 1.0. We learn beta but not the threshold, because learning both makes them redundant - scaling weights can compensate for either.

Q18. Reset by subtraction or reset to zero? Why?

Subtraction. Resetting to zero discards any potential above the threshold, so a strongly driven neuron loses information; subtracting V_th preserves the excess and keeps the firing rate proportional to the input.

Q19. What is the dead-neuron problem?

The spike function is a Heaviside step, whose derivative is zero everywhere except at the threshold, where it is undefined. Backpropagation through it produces zero gradient everywhere, so nothing can learn.

Q20. How do surrogate gradients solve it?

Forward pass uses the true hard step, so the network really is binary and spiking. The backward pass substitutes the derivative of a smooth approximation. We use fast_sigmoid with slope 25, so gradient flows for neurons near threshold.

Q21. What does the slope of 25 control?

How wide the gradient window around the threshold is. Too small and the gradient is diffuse and imprecise; too large and it approaches the true zero-almost-everywhere derivative and training stalls. 25 is the snnTorch convention and worked without tuning.

Q22. Isn't the surrogate gradient mathematically wrong?

It is a biased estimator of the true gradient, yes. It is justified empirically - it trains reliably - and theoretically as a smoothed relaxation of the discrete network. This is standard practice in the SNN literature, not something we invented.

Q23. What is rate coding? What other codings exist?

Rate coding puts information in spike count over a window - our approach. Alternatives are temporal or latency coding, where the time of the first spike carries the value, and population coding, where a group of neurons jointly encodes one quantity. Rate coding is the most robust to noise and the easiest to train.

Q24. Why 16 timesteps?

It is the accuracy-latency knee we measured. We tested 16, 32 and 64 and got 26.8%, 26.3% and 26.0% IoU on the earlier configuration - more timesteps bought nothing while multiplying latency, so we kept the cheapest.

Q25. What is spike sparsity and what did you measure?

The fraction of neurons that never fire across all timesteps. We measured about 29%, against a target of 85%. Our sparsity penalty weight of 0.05 is too gentle - that is an honest gap between our goal and our result.

Q26. Why does sparsity matter if it does not speed up your GPU?

It does not help us at all on a GPU, which pays for every multiply regardless of whether the operand is zero. Sparsity converts to energy saving only on event-driven hardware such as Loihi. We report it because it is the quantity that would matter on the intended deployment target.

Q27. So is your energy claim actually demonstrated?

No, and we say so directly. Our 105 ms on a GPU is slower than an equivalent CNN. The energy advantage is a property of neuromorphic hardware that we have not had access to. What we demonstrate is that the accuracy survives the spiking constraint.

Q28. What is the difference between an SNN and an RNN? Both have state.

An RNN carries a real-valued hidden vector and communicates real numbers between steps. An SNN carries an analog membrane potential but communicates single bits. That binary channel is exactly what makes event-driven hardware cheap.

Q29. Could you have converted a trained CNN into an SNN instead?

Yes - ANN-to-SNN conversion is a common route, and it usually needs hundreds of timesteps to match the original accuracy. We trained directly with surrogate gradients so that 16 timesteps would suffice.

Q30. Does Poisson encoding not add noise? Doesn't that hurt you?

We tested precisely that. Replacing the stochastic encoder with a noise-free analog input changed IoU from 26.0% to 23.4% - it did not help. The Poisson noise was never the bottleneck, which is why we stopped investigating the encoder.

C. Architecture (Q31-Q48)

Q31. Walk me through your architecture end to end.

Image 128x128x3 -> Poisson encoder -> spike train (16, B, 3, 128, 128) -> three spiking conv blocks -> layer-3 spikes (16, B, 128, 16, 16) plus final membrane potential -> ASAB attention bridge -> one fused map (B, 128, 16, 16) -> dense head -> 16x16 heatmap -> decode_mask -> bounding box and confidence.

Q32. What does the Poisson encoder actually do?

Clamps pixel intensity to [0, 1], treats it as a firing probability, and draws 16 independent Bernoulli samples per pixel. A pixel of 0.7 fires in roughly 11 of the 16 steps.

Q33. Describe the backbone.

Three blocks, each Conv2d -> BatchNorm -> Leaky LIF, taking channels 3 -> 32 -> 64 -> 128 while spatial resolution drops 128 -> 64 -> 32 -> 16. The whole stack is stepped 16 times with membrane state persisting across steps. 93 K parameters.

Q34. Why does the resolution stop at 16x16?

Three stride-2 stages from 128 px. It also matches the head's output grid, so no upsampling is needed anywhere in the network.

Q35. What is ASAB and why do you need it?

The Attention-based SNN-ANN Bridge. The backbone produces a tensor with a time axis; a detection head needs a single spatial map. ASAB collapses time using learned channel attention rather than a plain average, so informative timesteps count more.

Q36. How exactly does ASAB work?

It averages spikes over time, global-average-pools that to one value per channel, passes it through a 128 -> 32 -> 128 bottleneck with a sigmoid to get per-channel gates, multiplies the averaged spikes by those gates, concatenates the result with the final membrane potential, and fuses with a 1x1 convolution, BatchNorm and SiLU.

Q37. Why concatenate the membrane potential as well as the spikes?

Spikes are binary and lossy: a neuron sitting at 0.99 threshold and one at 0.01 both output zero. The membrane potential retains that sub-threshold analog evidence, which is exactly the faint signal a camouflaged object produces.

Q38. Is this attention the same as transformer attention?

No. There are no queries, keys or values and no token-to-token interaction. It is channel attention in the Squeeze-and-Excitation sense, applied over the temporal dimension.

Q39. Describe the detection head.

Two 3x3 convolutions at width 96 with BatchNorm and SiLU, then three parallel 1x1 convolutions producing a 1-channel heatmap, a 2-channel sub-cell offset and a 2-channel size map. In mask mode only the heatmap is used.

Q40. Why is the heatmap bias initialised to -4.595?

sigmoid(-4.595) = 0.01, so every cell starts out predicting 'almost certainly background'. Since most cells genuinely are background, this stops the loss being dominated by trivially correcting an over-confident initialisation. It is the focal loss prior trick.

Q41. How do you get a box out of the heatmap?

decode_mask thresholds the sigmoid heatmap, finds which rows and columns contain any above-threshold cell, and takes the tight rectangle spanning them. Cell indices convert to normalised image coordinates covering the full cell extent.

Q42. What is the threshold, and why is it adaptive?

min(0.10, peak * 0.5). The absolute part handles confident detections; the relative part means a faint but internally consistent detection still yields a full-extent box instead of a single cell.

Q43. How did you pick 0.10?

By sweeping it. We fed ground-truth masks in as logits and swept 0.05 to 0.50 over 300 samples. At the old default of 0.35 the decoder capped at 69.6% IoU; at 0.10 it reached 77.4% IoU and 93.1% IoU@50. That was an 11-point gain for zero training cost.

Q44. How is confidence computed?

The mean of the four highest cell probabilities in the grid. A single hot cell is noise; a real object lights up a contiguous region, so averaging the top four suppresses isolated false peaks.

Q45. Why not just take the maximum cell?

Because the maximum is the single noisiest statistic in the map. Any one spuriously over-fired cell would produce full confidence.

Q46. What are the offset and size heads for if mask mode ignores them?

They belong to the CenterNet-style path that decode() uses. We keep both decoders in the same class so the earlier centre-Gaussian checkpoints still load and the two supervision strategies remain directly comparable.

Q47. Why is the size head a liability in the centre-Gaussian path?

Because it regresses width and height, and regression against a clustered target collapses to the dataset mean. Ours sat at L1 0.21, exactly the value for predicting the mean. Reading size off the mask extent makes that failure mode structurally impossible.

Q48. Why is the dense head immune to the collapse the old head suffered?

The old head regressed one box from globally pooled features, so averaging two candidate positions was a valid low-loss compromise. In the dense head, averaging two candidate locations lowers the heatmap score at both - the compromise is punished rather than rewarded.

D. Datasets (Q49-Q60)

Q49. Which datasets did you use?

COD10K and NC4K. COD10K contributes 6,000 training and 4,000 test camouflaged images, plus 2,960 training and 1,974 test NonCAM negatives; NC4K contributes 2,884 training and 1,238 test images. In total 8,884 training and 5,238 test frames.

Q50. Why two datasets rather than one?

COD10K is the standard benchmark but is biased towards natural animal camouflage. NC4K adds different sources and scene statistics, so the combination reduces the risk of overfitting to one dataset's photographic conventions.

Q51. What are the NonCAM negatives and why do they matter?

Real scenes containing no camouflaged object. Without them every training sample would be positive, and a confidence head trained only on positives learns to output 1.0 always - which is exactly what our first model did.

Q52. Did you annotate anything by hand?

No. Both datasets ship pixel-accurate ground-truth masks. Our bounding boxes and 16x16 occupancy targets are derived deterministically from those masks, so the labels are reproducible and contain no human variance.

Q53. How do you build the 16x16 target?

Average-pool the binary ground-truth mask down to 16x16. Each cell's value is the fraction of its pixels belonging to the object, which we threshold into occupancy.

Q54. What augmentation did you use?

Horizontal flip and mild scale-and-crop jitter.

Q55. Why no colour jitter or brightness augmentation?

Because colour and texture similarity between the target and its background is the signal we are detecting. Perturbing hue and saturation would destroy the exact cue the task depends on. This is a task-specific decision, not laziness.

Q56. Why did you resize to 128x128? Isn't that small?

It is small, and it costs us fine detail. It was forced by memory: the spike tensor is (16, B, C, H, W), so activation memory scales with the timestep count. At 4 GB of VRAM, 128 px with batch 16 was what fit.

Q57. How did you split train and test?

We used the datasets' own published Train and Test splits, so our numbers are comparable to published work and there is no chance of us having leaked test images into training.

Q58. Is your test set balanced?

No. Test contains 5,238 frames of which 1,974 are negatives, so roughly 38% negatives. IoU is computed only over positive frames; confidence separation is what measures behaviour on negatives.

Q59. Do you handle multiple objects in one image?

Not currently. decode_mask returns one rectangle spanning all above-threshold cells, so two animals produce one box covering both. Connected-component labelling on the mask would fix this and is our planned next step.

Q60. Is there class imbalance in your supervision?

Yes - objects occupy roughly 25% of the grid on average, and about a third of frames are pure background. That is why we combine BCE with Dice loss, which is scale-invariant and prevents small objects from being ignored.

E. Training (Q61-Q75)

Q61. What loss function did you use?

BCE with logits plus soft Dice on the 16x16 heatmap, weighted 1.0 and 1.0, plus a one-sided sparsity penalty weighted 0.05.

Q62. Why both BCE and Dice? Isn't one enough?

BCE alone is dominated by the background majority, so a model predicting all background scores well. Dice is a ratio of overlap to total area, so it is scale-invariant and cares about small objects. Together they give stable gradients for both classes.

Q63. Write your Dice formula.

dice = 1 - (2 * intersection + 1) / (sum(p) + sum(t) + 1), computed per image on the flattened grid. The +1 terms are Laplace smoothing, which keeps the expression defined when a frame is pure background and both sums are zero.

Q64. What is the sparsity penalty?

relu(0.85 - measured_sparsity), so it only applies when the network is less sparse than the 85% target and never penalises being sparser. Weight 0.05.

Q65. What optimiser and schedule?

AdamW at learning rate 2e-3 with weight decay 1e-4, and a OneCycleLR schedule with pct_start 0.3. Gradients are clipped at norm 5.

Q66. Why gradient clipping?

Backpropagation through 16 timesteps compounds gradients, much like an RNN, so occasional spikes in gradient norm are expected. Clipping stops one bad batch from destroying the run.

Q67. Why OneCycleLR?

The warm-up avoids destabilising the network before BatchNorm statistics settle, and the long decay acts as an implicit fine-tune at the end. It is also a strong default that needs almost no tuning, which mattered with our compute budget.

Q68. What batch size and why?

16, which is the largest that fit in 4 GB given the 16-timestep activation tensor.

Q69. How many epochs did you train?

Four. Validation loss was still falling when we stopped, so we are undertrained and we say so - that is our single biggest weakness, and it is a budget limitation rather than a design one.

Q70. How do you validate, and why on a subset?

We evaluate on a strided 640-frame subset of the test split each epoch. Full validation runs 16 timesteps over 5,238 frames, which would have dominated epoch time. The stride is deterministic, so the subset is unbiased and consistent across epochs. Final numbers are reported on the full split.

Q71. Which checkpoint do you keep?

The one maximising iou + 0.5 * separation, not IoU alone.

Q72. Why that composite score?

Because IoU is computed only over positive frames. A model that emits one large box on every image, including empty ones, can score well on IoU while being useless. Adding separation forces it to also distinguish present from absent.

Q73. What is the resume mechanism, and what could go wrong with it?

NEUROCAMO_RESUME loads existing weights. Two hazards: a fresh OneCycle at full max_lr would destroy the resumed solution during warm-up, so we cut peak learning rate to 30%; and a best-score initialised at -1 would let a bad first epoch overwrite good weights, so we seed it from the resumed checkpoint's own score.

Q74. Did you use pretrained weights?

No. Training is from scratch. There are no ImageNet-pretrained spiking backbones available, which is a real disadvantage against any ANN baseline and part of why our absolute numbers are modest.

Q75. How did you verify the architecture could learn at all before blaming supervision?

We overfitted 24 images and reached 76% IoU. That proves the gradient path and architecture are sound, so the failure on the full set had to be supervision or capacity - which is what directed us to mask supervision.

F. Results and metrics (Q76-Q86)

Q76. What is IoU?

Intersection over union: the area shared by the predicted and true boxes divided by the area either covers. 1.0 is a perfect box, 0 is no overlap at all.

Q77. What is IoU@50?

The fraction of positive frames whose IoU is at least 0.5 - the conventional detection-success criterion. Ours is 33.7%, so roughly one frame in three is a usable detection.

Q78. What is the mean-box baseline and why do you report it?

Predict the average ground-truth box on every image, ignoring the input entirely. On our test split that scores 37.0% IoU. We report it because without it, a number like 38.6% cannot be distinguished from having learnt nothing at all.

Q79. Then you only beat the baseline by 1.6 points. Isn't that damning?

On IoU alone it looks weak, and we do not hide it. But the baseline is a fixed box, so its IoU@50 is near zero and its confidence separation is exactly zero. Ours are 33.7% and +0.288. The baseline scores well on mean IoU precisely because that metric is forgiving on the centred, medium-sized objects these datasets favour.

Q80. What is confidence separation?

Mean confidence on frames containing a target minus mean confidence on frames without one. It went from +0.003 in our first model to +0.288 - a 96-fold improvement, and it is what actually fixed the visible false positives.

Q81. Give me the before-and-after comparison.

Global-MLP head: 9.6% IoU, ~2% IoU@50, +0.003 separation. Same architecture with centre-Gaussian dense supervision: 26.3%, ~7%, +0.011. Mask supervision: 38.6%, 33.7%, +0.288.

Q82. What is the decoder ceiling test, and what did it show?

We fed ground-truth masks in as if they were the network's logits and decoded them. That isolates the decoder from the network and gives 77.4% IoU and 93.1% IoU@50. So the decoder is not the bottleneck, and 38.6% is roughly half of what this exact architecture can reach.

Q83. Why are your boxes sometimes too large?

Grid quantisation. A 16x16 grid over 128 pixels means each cell is 8 pixels, so one spuriously over-fired edge cell widens the box by about 6%. Combined with undertraining, the mask leaks slightly beyond the object.

Q84. How would you fix the loose boxes?

Three ways, in order of cost: train to convergence, raise the grid to 32x32 so cells are 4 pixels, and add a boundary-aware term to the loss that penalises mask leakage specifically at object edges.

Q85. What was the label-contamination bug?

Our fallback bounding box for frames with no object was non-zero, which made the has_object flag true for negatives. The model was being told that empty scenes contained targets, so it learnt to say yes always. A one-line fix, and it revealed the real 9.6%-versus-32.5% picture that had been hidden.

Q86. What lesson did you take from that bug?

That a suspiciously confident model is a labelling hypothesis before it is an architecture hypothesis. It also made us report the baseline on everything, because the baseline is what exposed the problem.

G. Comparison with other models (Q87-Q92)

Q87. How is this different from YOLO?

YOLO is a non-spiking CNN with real-valued activations, one forward pass, 7-70 M parameters, and anchor or anchor-free box regression, usually with COCO pretraining. Ours is spiking with binary activations, 16 sequential timesteps, 0.33 M parameters, trained from scratch, with the box read from a predicted occupancy mask rather than regressed.

Q88. Would a YOLO fine-tuned on COD10K beat you?

Almost certainly, and we should say so before the panel does. It has 20 to 200 times the parameters and the benefit of large-scale pretraining. Our claim is not higher accuracy than YOLO; it is a camouflage detector that runs on event-driven hardware at 0.33 M parameters - a different point on the accuracy-energy curve.

Q89. How do you compare against dedicated COD networks like SINet or SegMaR?

They reach far higher accuracy, in the 80% range on these benchmarks, with roughly a hundred times our parameters and full segmentation output. They are the accuracy reference point; we are not competing with them on that axis.

Q90. Could you not get the same result with a small plain CNN?

Very likely yes, on a GPU, and probably faster. That is a fair criticism and the honest answer is that the spiking formulation is justified only by the neuromorphic deployment target. On a GPU it is a cost, not a benefit.

Q91. Why not use SAM or a foundation model?

SAM is hundreds of millions of parameters and needs a prompt; it segments what you point at rather than deciding whether something concealed is present. It is also far outside any edge power budget, which is the constraint the whole project is built around.

Q92. What would you compare against if you had more time?

A parameter-matched non-spiking CNN, trained identically. That is the only comparison that isolates the cost of the spiking constraint, and it is the experiment our project is currently missing.

H. Dashboard and code (Q93-Q97)

Q93. What does the spike spectrogram show?

A 128-channel by 16-timestep matrix of layer-3 firing rates, averaged over the 16x16 spatial grid. Brightness is that channel's firing rate at that timestep. The left edge is dim because membranes are still charging, which is exactly the temporal imbalance ASAB learns to correct.

Q94. Is the heatmap overlay a saliency method like Grad-CAM?

No, and it is stronger than one. It is the actual tensor the box was decoded from, not a post-hoc reconstruction, so the overlay cannot disagree with the prediction.

Q95. What are the Laplacian variance and the scene gates? Are they part of the network?

No - they are classical OpenCV heuristics. They existed only because the old head emitted 0.998 confidence on every input, so a hand-written filter was the only thing separating a hit from a miss. With real objectness from the dense head they are bypassed, because they were destroying correct detections in flat or bright scenes.

Q96. How does the web app relate to the research code?

inference.py adds the research folder to sys.path and imports the same encoder and model classes, so there is no duplicated model code. It reads the checkpoint's head field to build the correct class before loading weights, and keeps the model in one long-lived bundle so start-up cost is paid once.

Q97. Is the web app secure?

It binds 127.0.0.1 only, so it is not reachable from the network, and uploads are size-capped with a clean 413 response. But /api/detect has no authentication, which is fine for a local demo and would need a token and TLS before any deployment.

I. The hard questions (Q98-Q100)

Q98. Your project's motivation is energy efficiency, but you never measured energy, and your model is slower than a CNN. Isn't the premise unsupported?

On our hardware, the energy claim is unsupported and we do not assert it. What we have shown is the necessary precondition: that a 0.33 M-parameter spiking network can learn camouflaged detection at all, which is what a neuromorphic deployment would need. The joules-per-frame measurement requires Loihi or SpiNNaker access, and it is the first item of future work rather than a result we are claiming.

Q99. If supervision density was the whole problem, is the spiking part contributing anything?

Supervision density is what unlocked learning; it does not tell us the spiking backbone is optimal. The honest position is that we have shown the spiking pipeline is *sufficient* for the task and have not shown it is *necessary*. The parameter-matched non-spiking control is the experiment that would settle it, and we have not run it.

Q100. What is the single biggest weakness of this project, in your own words?

Four epochs. We stopped while validation loss was still falling, and our own decoder ceiling test says this exact architecture can reach 77.4% IoU. So roughly half our remaining error is unspent training budget, not a design flaw - which is simultaneously the most embarrassing and the most fixable thing about the work.

How to answer anything not on this list

- Quote only measured numbers. If you do not know a figure, say you would have to check rather than estimating out loud.
- If a criticism is correct, concede it in one sentence and then say what the fix is. A conceded weakness with a plan reads as competence; a defended weakness does not.
- Bring every answer back to the design point: 0.33 M parameters, event-driven hardware, camouflaged targets.
- If asked something you have genuinely not considered, say so and reason aloud from the architecture. Panels grade reasoning, not recall.